PDF	PDF	PDF	PDF	PDF	PDF
1	2	3	4	5	6

I have uploaded 8 research papers on trajectory prediction, motion forecasting, autonomous driving, planning, and related topics.
Your role is NOT to summarize these papers.

Modern Trajectory Prediction for Autonomous Driving: A Graduate-Level Handbook

Welcome to the motion prediction team. This handbook is designed to take you from first principles to the bleeding edge of modern trajectory prediction, enabling you to understand, critique, and implement state-of-the-art architectures from scratch.

PART 1 — Foundations of Trajectory Prediction

What is trajectory prediction?

Trajectory prediction, or motion forecasting, is a core component of modern autonomous driving systems. It involves processing the observed historical states of surrounding agents (such as vehicles, pedestrians, and cyclists) along with the complex topological information of the road map to accurately anticipate their future spatial coordinates over a designated time horizon. The goal is to provide the ego vehicle's planning module with actionable foresight regarding how the traffic scene will evolve. PDF + 1

Why is it difficult?

The difficulty of motion forecasting stems from several compounding factors:

Diverse and Multimodal Behaviors: Traffic participants do not follow a single deterministic path; a vehicle approaching an intersection might go straight, turn left, or turn right based on latent long-term goals. PDF + 2

Complex Interactions: Agents heavily influence one another. A pedestrian crossing the street will cause an approaching vehicle to decelerate, creating a highly coupled joint distribution of future states. PDF + 1

Environmental Intricacies: The road network contains complex topologies, traffic rules, and semantic priors that dictate feasible motion. PDF + 1

Streaming Context: Real-world driving is continuous, requiring models to efficiently process streaming data without redundant re-computations. PDF

Why do deterministic models fail?

Early approaches often relied on deterministic decoders, such as simple Multi-Layer Perceptrons (MLPs), to regress a single future trajectory from encoded scene features. These methods inherently fail because they are penalized for predicting anything other than the exact ground truth. Consequently, during optimization, the model learns to output the mean of all possible futures. This yields homogeneous, averaged trajectories that often drift off the drivable road surface or fail to capture the true, distinct modalities of the agent's intent. PDF + 2

Why is multi-modal prediction necessary?

Because the future is unobserved and heavily dependent on the unknown intentions of human drivers, the output of a prediction model must represent a probability distribution over multiple plausible futures. Predicting multi-modal trajectories allows the autonomous vehicle to prepare for various contingencies. This is commonly achieved by predicting a discrete set of trajectories, each associated with a confidence score or probability. PDF + 3

What is uncertainty? (Aleatoric vs. Epistemic)

While not always explicitly partitioned in engineering applications, uncertainty in prediction is twofold:

Aleatoric Uncertainty: The inherent randomness or unpredictability in the environment. Even with perfect observations, an agent's exact future path has variance (e.g., slight lateral deviations within a lane). This is modeled using continuous distributions like Gaussian Mixture Models (GMMs) for trajectory waypoints. PDF

Epistemic Uncertainty: Uncertainty arising from the model's lack of knowledge or limited training data. This is typically addressed by scaling datasets and improving architectural capacity.

Open-loop vs Closed-loop evaluation

Open-loop evaluation: The model's predictions are compared directly against pre-recorded, logged expert trajectories. The environment is static, and the ego vehicle does not actually execute a plan to see how the world reacts. PDF

Closed-loop evaluation: Evaluates the planner in a simulated environment where the autonomous vehicle executes its decisions step-by-step. Metrics evaluate safety (collision rates), route progress, and kinematic comfort (acceleration and jerk). PDF

Reactive vs Proactive planning

Reactive Planning: The autonomous vehicle receives a fixed set of predictions for all agents and plans a collision-free path around them. It assumes its own actions will not alter the future behaviors of surrounding agents.

Proactive (Interactive) Planning: The autonomous vehicle recognizes that its decisions will influence other drivers. It actively coordinates by forecasting how agents will respond to its own hypothetical plans, a necessity for maneuvers like unprotected left turns or merging. PDF

Prediction vs Planning

Prediction: The task of forecasting the future states of *other* agents in the environment. PDF

Planning: The task of generating the *ego vehicle's* intended future trajectory and corresponding control actions. PDF

Motion forecasting vs Behavior prediction

Motion Forecasting: Outputs fine-grained, continuous geometric coordinates (e.g., x, y points) over a future time horizon. PDF

Behavior Prediction: Outputs discrete, high-level semantic intentions (e.g., lane change, yield, turn). PDF

PART 2 — Evolution of the Field

To understand the current state-of-the-art, we must trace how the field has evolved to overcome successive bottlenecks.

Era 1: Rasterization and Convolutional Networks

Early works rendered the driving scene (map and agent history) into multi-channel bird's-eye view (BEV) images. These images were processed using standard Convolutional Neural Networks (CNNs). PDF + 1

Limitation: Rasterization was highly lossy, destroying structural topological information, and required a massive computational and memory footprint due to the large receptive fields needed. PDF + 1

Era 2: Vectorization and Graph Neural Networks

To solve the inefficiency of rasterization, researchers shifted to vectorized representations (e.g., VectorNet). Maps and trajectories were represented as polylines or graphs. PDF + 1

Limitation: While GNNs efficiently aggregated sparse information, modeling long-range dependencies across complex intersections remained difficult. PDF

Era 3: Transformer Encoder-Decoders

Transformers replaced GNNs due to their superior capability in capturing global interrelationships among scene elements using self-attention and cross-attention. PDF

> **Limitation:** Standard agent-centric Transformers required re-encoding the entire scene for every agent, leading to $\mathcal{O}(N)$ redundant computations when predicting multiple agents. PDF + 1

Era 4: Query-Centric and Symmetric Architectures

To achieve multi-agent efficiency, models like **MTR++** and **QCNet** introduced query-centric or symmetric coordinate systems. By encoding scene elements in their local frames and handling spatial relationships through relative positional embeddings, the scene encoding could be computed once and shared across all agents. PDF + 4

> **Limitation:** While efficient, decoding multi-modal trajectories still heavily relied on heuristic anchor sets or dense goal candidates, which struggled with long-term predictions. PDF + 1

Era 5: Generative Models (Diffusion)

To better model the highly complex, continuous distribution of future behaviors without relying on fixed anchors, generative approaches like **MotionDiffuser** and **DiffusionDrive** applied denoising diffusion probabilistic models to trajectory generation. PDF + 1

> **Limitation:** Vanilla diffusion requires excessive denoising steps, hindering real-time latency. PDF

Era 6: Unified Policy World Models

Most recently, research focuses on bridging the gap between world simulation and trajectory planning. **Policy World Model (PWM)** demonstrates that unifying world modeling (action-free video forecasting) and planning allows the agent to utilize "anticipatory perception" for safer decision-making. PDF + 1

PART 3 — Deep Paper Analysis

Chapter 1: MTR & MTR++ (Motion Transformer)

Motivation

Traditional goal-based prediction methods rely on a dense set of goal candidates to capture multi-modal futures. However, excessive candidates drastically increase computational and memory costs, while too few degrade performance. MTR solves this by introducing a small set of learnable intention queries. Furthermore, MTR++ addresses the inefficiency of single-agent-centric modeling by introducing symmetric scene context modeling to predict multiple agents concurrently. PDF + 4

High-Level Intuition

Instead of guessing destinations from thousands of points, MTR uses 64 "intention queries" generated via k-means clustering on training data. Each query is responsible for a specific motion mode (direction and velocity). The model localizes the agent's global intention (which query is most likely) and simultaneously refines the local movement (the exact trajectory points). PDF + 2

Architecture

1. **Input Representation:**
   Agent histories and road maps are vectorized as polylines.
   History states are $S_A^{(g)} \in \mathbb{R}^{N_a \times T_h \times C_a}$.
   Map polylines are $S_M^{(g)} \in \mathbb{R}^{N_m \times n \times C_m}$. PDF + 2
2. **Polyline Encoder:**
   A PointNet-like MLP and max-pooling extract token features for agents ($F_A^{(g)}$) and maps ($F_M^{(g)}$). PDF
3. **Local Transformer Encoder:**
   Instead of global attention, a local multi-head self-attention restricts attention to a token's k nearest neighbors to preserve topological locality and reduce memory. PDF + 1
4. **Dense Future Prediction (Auxiliary):**
   A regression head predicts future states for *all* agents directly from history features to enhance the context provided to the focal agent. PDF
5. **Motion Decoder with Intention Queries:**
   64 intention points $I^{(s)} \in \mathbb{R}^{K \times 2}$ serve as anchors.
   Their positional embeddings form the learnable queries $E_I^{(s)} \in \mathbb{R}^{K \times D}$.
   The queries undergo self-attention (mode-to-mode) and cross-attention (mode-to-scene). PDF + 3
6. **Dynamic Map Collection:**
   To refine trajectories, the cross-attention dynamically restricts its keys/values to map polylines closest to the currently predicted trajectory of that specific query. PDF
7. **Symmetric Scene Modeling (MTR++ only):**
   Encodes polylines in their own local coordinate systems, outputting $F_A^{(l)}$ and $F_M^{(l)}$. It relies on a "query-centric self-attention" where relative coordinates are computed on the fly. PDF + 2
8. **Mutually-Guided Intention Querying (MTR++ only):**
   Intention queries from multiple different agents interact via self-attention to generate scene-compliant, non-colliding joint trajectories. PDF

Mathematical Formulation

Query-Centric Relative Transformation (MTR++)
To compute attention symmetrically without a global anchor, the relative position $R_{AM}^{(pos)}[i,j]$ and heading $R_{AM}^{(ang)}[i,j]$ of token j with respect to query token i is calculated: PDF

$$R_{AM}^{(pos)}[i,j] = (P_{AM}^{(g)}[j] - P_{AM}^{(g)}[i]) \begin{bmatrix} \cos H_{AM}^{(g)}[i] & -\sin H_{AM}^{(g)}[i] \\ \sin H_{AM}^{(g)}[i] & \cos H_{AM}^{(g)}[i] \end{bmatrix}$$

PDF

Output GMM Distribution

The network outputs parameters for a Gaussian Mixture Model. For time step i, the spatial probability density at location $o \in \mathbb{R}^2$ is:

PDF

$$\mathcal{P}_i(o) = \sum_{k=1}^{K} p_k \cdot f_k(o_x - \mu_x, o_y - \mu_y)$$

PDF

K: Number of components (64 queries). PDF

p_k: The classification probability of the k-th mode. PDF

f_k: The Gaussian probability density function predicted by query k. PDF

Loss Functions

1. **Negative Log-Likelihood (L_{GMM}):**

$$L_{GMM} = -\log f_h(\hat{Y}_x - \mu_x, \hat{Y}_y - \mu_y) - \log(p_h)$$

Intuition: Maximizes the likelihood of the ground-truth position $(\hat{Y}_x, \hat{Y}_y)$. Only the "positive" component h (the query closest to the ground-truth endpoint) is penalized.

PDF + 2

2. **Dense Future Prediction Loss (L_{DMP}):**
An L1 regression loss applied to the auxiliary dense future prediction of all agents.

PDF

Training Pipeline

Optimizer: AdamW. PDF

Learning Rate: 0.0001 with weight decay 0.01. PDF

Batch Size: 80 scenes across 8 GPUs. PDF

Epochs: 30 epochs, with LR decayed by 0.5 every 2 epochs starting from epoch 20.

PDF

Evaluation

MTR achieves state-of-the-art on the Waymo Open Motion Dataset (WOMD). PDF

mAP (Mean Average Precision): Emphasizes the confidence scoring of the trajectories. MTR++ achieves 0.4329 mAP. PDF

Miss Rate: The ratio of scenarios where the best predicted trajectory's endpoint is more than 2.0 meters from the ground truth. MTR++ achieves 12.98%. PDF

minADE/minFDE: Minimum Average/Final Displacement error of the top K predictions.

Ablation Studies

Learnable queries vs. Dense goals: Using k-means generated intention queries yielded an mAP of 0.3539, outperforming dense goals (0.2912) and latent embeddings (0.2826) . PDF

Uniform vs k-means: K-means clustering for intention points drastically reduces Miss Rate (15.75%) compared to uniform grids (19.52%). PDF

Strengths & Weaknesses

Strengths: Highly efficient due to a small number of queries (64). MTR++ provides exceptional multi-agent efficiency. PDF + 1

Weaknesses: Relies heavily on accurate quality score estimation. The model often struggles to rank the best trajectory highest, leading to an mAP drop. It can also output homogenized trajectories in rare scenarios, failing to capture true behavioral diversity. PDF + 1

Stopping here due to length constraints. Awaiting your 'Continue' command to proceed to the next section.

Continue

Chapter 2: DenseTNT (End-to-End Trajectory Prediction from Dense Goal Sets)

Motivation

Before DenseTNT, goal-based prediction models typically used a sparse set of goal candidates (e.g., generated by heuristics or clustering). The problem was that if the true destination wasn't included in the sparse set, the model's accuracy suffered drastically. Alternatively, regression-based models suffered from mode collapse, predicting an average trajectory that didn't accurately represent any distinct intention. DenseTNT aimed to solve this by densely sampling a massive number of goal candidates across the map and learning to select the best ones while avoiding the computational bottleneck typically associated with dense sampling.

High-Level Intuition

Imagine dropping thousands of pins on the drivable area of a map. The model first scores the probability of the agent arriving at *each* pin. Then, instead of returning thousands of trajectories, it uses a smart algorithm to select a diverse subset of the most likely pins. Finally, it completes a full trajectory for each selected pin. This guarantees that all potential destinations are considered without losing the diversity of multimodal behaviors.

Architecture

1. **Input Representation:**
   The scene is represented as a set of polylines using VectorNet. Agent histories and map lanes are encoded as nodes in a graph. PDF + 2

2. **Dense Goal Candidates:**
   The model generates a dense grid of goal candidates across the drivable area of the map. PDF + 2

3. **Context Encoding:**
   A hierarchical graph neural network (like VectorNet) encodes the relationships between agent nodes and map nodes. This yields a rich, global context feature for the scene. PDF + 2

4. **Goal Scoring (Endpoint Prediction):**
   For every goal candidate, the model predicts the probability that it is the agent's final destination. This is essentially a dense classification task over the map.

5. **Trajectory Completion:**
   Given a goal candidate, a trajectory completion module (typically an MLP or recurrent network) generates the intermediate waypoints leading from the agent's current position to that specific goal. PDF + 2

6. **Optimization (The "Dense" Part):**
   The core innovation is how DenseTNT handles the sheer volume of candidates. During training, instead of just penalizing the single closest goal, it uses a soft-labeling technique or an optimal transport formulation to distribute the probability mass among nearby candidates, ensuring smooth and stable learning across the dense grid.

Mathematical Formulation

Goal Probability Distribution
Let $G = \{g_1, g_2, \dots, g_M\}$ be the dense set of goal candidates. The model predicts a probability distribution $P(g|X)$ over these candidates given the scene context X.
The training loss for goal prediction typically minimizes the cross-entropy between the predicted distribution and a target distribution (which is peaked around the ground-truth endpoint).

Training Pipeline

Losses: A combination of goal classification loss (cross-entropy) and trajectory completion loss (L1 or Huber loss on waypoints). PDF + 2

Optimization: Adam optimizer. PDF + 2

Evaluation

DenseTNT achieved top-tier performance on the Waymo Open Motion Dataset (WOMD) at the time of its publication. PDF + 2

Strengths: Exceptional performance in capturing the true endpoint distribution due to the dense coverage. It effectively mitigates the issue of missing the true goal. PDF + 2

Weaknesses: Computationally heavy. Evaluating thousands of goals and running non-maximum suppression (NMS) or similar selection algorithms is slow. As shown in Table 3 of the MTR paper, DenseTNT has an inference latency of 540ms, making it impractical for real-time systems compared to query-based methods (which run in ~60-120ms). PDF + 4

Chapter 3: Scene Transformer

Motivation

Traditional models (like VectorNet or early goal-based models) are "agent-centric." To predict the future for Agent A, the entire coordinate system is centered and rotated relative to Agent A. If you want to predict futures for Agent B, you must re-center the map, re-encode the polylines, and re-run the entire network. This $\mathcal{O}(N)$ scaling is prohibitively slow for scenes with many interacting agents. Scene Transformer was designed to predict the trajectories of *all* agents in the scene simultaneously using a single, unified representation. PDF + 2

High-Level Intuition

Why re-compute the map for every car? Scene Transformer encodes the scene once. It uses a "scene-centric" approach where the relationships between agents and the map are modeled using a massive Transformer network. By factorizing the attention mechanism (computing attention across time, then across agents, then across map elements), it handles the complexity without running out of memory.

Architecture

1. **Input Representation:**
   Agents and map elements are represented as a set of tokens. PDF + 2
2. **Factorized Attention:**
   A standard Transformer's self-attention scales quadratically with the number of tokens. If a scene has A agents, M map elements, and T timesteps, computing self-attention over all $(A + M) \times T$ tokens is intractable. Scene Transformer solves this by factorizing the attention: PDF + 1

   Temporal Attention: Attention across time T for each agent independently. PDF + 1

   Agent-Map Attention: Attention between agents and map elements at the current timestep. PDF + 1

   Social (Agent-Agent) Attention: Attention among all agents to capture interactions. PDF + 1
3. **Unified Decoding:**
   The network outputs predictions for all agents simultaneously. PDF + 2

Evaluation & Limitations

Strengths: Highly scalable. It allows for joint prediction of multiple agents without the linear scaling cost of agent-centric models. PDF + 2

Weaknesses: Despite being scene-centric, the global coordinate system still creates biases. Agents far from the origin (often the ego-vehicle) might suffer degraded performance because the shared context distribution is uneven. It also struggles to achieve the very low miss rates of goal-based or query-based methods. PDF + 1

Chapter 4: QCNet (Query-Centric Trajectory Prediction)

Motivation

While Scene Transformer allowed multi-agent prediction, it still relied on a global coordinate system, leading to the biases mentioned above. Furthermore, in an online, continuous driving system, the vehicle receives data frame-by-frame. Agent-centric models must re-compute everything when the coordinate frame moves. QCNet introduces a "query-centric" encoding paradigm to achieve spatially invariant representations, enabling streaming (online) prediction where past computations can be cached and reused. PDF

High-Level Intuition

Instead of defining the world relative to one global center, QCNet defines a *local* coordinate system for *every single element* (every agent state, every map polygon). When a query needs to gather context, it transforms the surrounding elements into its own local frame on the fly using relative positional embeddings. Because the raw features of a map polygon or an agent's past state are computed in their own local frames, they are completely invariant to where the ego-vehicle is currently located. Thus, as the car drives forward, the encoded features from $t-1$ are still valid at t and can be reused from a cache. PDF

Architecture

1. **Local Spacetime Coordinate System:**
   For the i-th agent at time t, the local frame is defined by its position p_i^t and heading θ_i^t. Map polygons use their entry point position and orientation. PDF
2. **Scene Element Embedding:**
   All geometric attributes (e.g., velocity, sampled points on a lane) are computed relative to the element's *own* local frame. These are transformed using Fourier features to capture high-frequency signals, concatenated with semantic attributes, and passed through an MLP to create invariant embeddings (Agent shape: $[A, T, D]$, Map shape: $[M, D]$). PDF
3. **Relative Spatial-Temporal Positional Embedding:**
   To allow elements to "talk" to each other, QCNet computes a 4D descriptor summarizing their relative position: distance, relative direction, relative orientation, and time difference. This descriptor is passed through an MLP to create a relative positional embedding $r_{j \to i}$. PDF
4. **Factorized Attention (Streaming Encoder):**
   The encoder uses factorized attention (Temporal, Agent-Map, Social). Because everything is invariant, in an online setting, when a new frame arrives, QCNet only computes attention for the *new* incoming agent states ($\mathcal{O}(A^2)$ complexity) rather than re-computing the entire T history ($\mathcal{O}(AT^2)$), drastically reducing latency. PDF
5. **Query-Based Decoder (Anchor-Free + Anchor-Based):**
   QCNet employs a two-stage decoder to handle the massive uncertainty of long-term prediction: PDF

   Mode2Scene & Mode2Mode Attention: Similar to MTR, queries interact with the scene and each other. PDF

   Anchor-Free Trajectory Proposal (Recurrent): Instead of predicting the whole 6-second future at once, queries decode a short segment, then recurrently re-

query the scene context to predict the next segment. This handles changing environments. PDF

Anchor-Based Refinement: The proposals from the first stage act as dynamic anchors. A second refinement module refines these proposals and predicts final probabilities. PDF

Mathematical Formulation

Relative Positional Descriptor
For element i at (p_i^t, θ_i^t, t) and element j at (p_j^s, θ_j^s, s), the 4D descriptor is:

$$[||p_j^s - p_i^t||_2, \text{atan2}(p_{j,y}^s - p_{i,y}^t, p_{j,x}^s - p_{i,x}^x) - \theta_i^t, \theta_j^s - \theta_i^t, s - t]$$

This descriptor is embedded and added to the Key and Value vectors during attention, injecting spatial awareness without requiring a global coordinate frame. PDF

Training Pipeline

Losses: Modeled as a mixture of Laplace distributions.
PDF

$$f(y_i^{1:T}) = \sum_{k=1}^{K} \pi_{i,k} \prod_{t=1}^{T} \text{Laplace}(\mu_{i,k}^t, b_{i,k}^t)$$

The model uses a winner-take-all strategy, backpropagating regression loss only through the best predicted proposal and refinement. A classification loss optimizes the mixing coefficients π. PDF

Evaluation
QCNet achieved 1st place on Argoverse 1 and Argoverse 2 benchmarks. PDF

Efficiency: The streaming encoder reduces latency drastically. For 190 agents and 169 map polygons, inference drops from 82ms (without reuse) to 13ms (with reuse). PDF

Strengths: Unparalleled efficiency in continuous, online settings. The recurrent decoder handles long-term prediction (e.g., 6 seconds in Argoverse 2) exceptionally well. PDF

Weaknesses: The architecture is highly complex to implement due to the intricate relative coordinate transformations and caching mechanisms required for streaming.
PDF

Chapter 5: MotionDiffuser (Diffusion Models for Multi-Agent Prediction)

Motivation
While models like MTR and QCNet achieve high performance, they still rely on discrete queries or anchors to represent the multi-modal distribution. This inherently limits the diversity and resolution of the predictions. Diffusion models, which have revolutionized

image generation by learning to sample from continuous, highly complex distributions, offer a compelling alternative. MotionDiffuser adapts Denoising Diffusion Probabilistic Models (DDPMs) to predict the joint motion of multiple agents without relying on predefined anchors. Furthermore, diffusion models allow for *guided sampling* at inference time, enabling the injection of rules (e.g., "do not collide") without retraining. PDF

High-Level Intuition

Instead of predicting trajectories directly, the model learns to *denoise*. During training, we take ground-truth trajectories and slowly add Gaussian noise until they are complete static. The neural network is trained to reverse this process: given a noisy trajectory and the scene context (map, history), predict the slightly less noisy trajectory. During inference, we start with pure random noise and iteratively denoise it, conditioned on the scene, to generate physically plausible trajectories. Because we start with different random noise each time, we naturally generate a diverse, multi-modal distribution of futures. PDF

Architecture

1. **Scene Context Encoding:**
   Similar to previous models (e.g., Wayformer backbone), historical states, traffic lights, and road graphs are encoded into a set of context tokens C. PDF
2. **PCA Trajectory Representation:**
   Crucial Innovation: Predicting sequences of x, y coordinates (e.g., 80×2 for 8 seconds at 10Hz) directly with diffusion is inefficient and noisy. MotionDiffuser compresses trajectories using Principal Component Analysis (PCA).
   They found that just 10 PCA components can reconstruct an 80×2 trajectory with only 0.06m error. The diffusion process operates on these compressed PCA latents ($S \in \mathbb{R}^{N_a \times N_p}$), not the raw coordinates. This vastly improves speed and stability. PDF
3. **Permutation-Invariant Set Denoiser:**
   The denoiser $D_\theta(S; C, \sigma)$ must handle multiple agents concurrently. PDF

   Input: Noisy PCA trajectories S, concatenated with a Fourier-embedded noise level σ. PDF

   Transformer Blocks: Repeated blocks of Self-Attention (across the set of agents to model joint interactions) and Cross-Attention (between agents and the scene context tokens C). PDF

   Because there is no positional encoding along the agent dimension, the network is permutation-equivariant, meaning the order of agents doesn't matter. PDF

Mathematical Formulation

Denoising Objective

The network D_θ is trained to predict the clean trajectory x from a noisy observation $x + \epsilon$:

$$\arg\min_\theta \mathbb{E}_{x \sim p_x} \mathbb{E}_{\sigma \sim q(\sigma)} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, \sigma^2 I)} || [cite_start] D_\theta(x + \epsilon; c, \sigma) - x ||_2^2$$

PDF

Constrained Sampling (Guidance)
This is the most powerful feature of MotionDiffuser. Suppose we want to sample from the joint distribution $p(S|C)$ but enforce a constraint $q(S|C)$ (e.g., "avoid collisions"). We need the modified score function:

PDF

$$\nabla_S \log(p(S|C) \cdot q(S|C)) = \nabla_S \log p(S|C) + \nabla_S \log q(S|C)$$

Since D_θ approximates the clean trajectory even at high noise levels, the constraint gradient can be approximated using an arbitrary differentiable cost function $\mathcal{L}$ applied to the denoised prediction:

PDF

$$\nabla_S \log q(S|C, \sigma) \approx \lambda \frac{\partial}{\partial S} \mathcal{L}(D_\theta(S; C, \sigma))$$

PDF

Attractor Cost: Forces the endpoint of a trajectory to reach a specific target. PDF

Repeller Cost: Penalizes trajectories where interacting agents get too close, mitigating collisions. PDF

Training Pipeline

Dataset: Waymo Open Motion Dataset (Interactive Split). PDF

Optimizer: AdamW, LR 5×10^{-4} with 10k warmup steps. PDF

Hardware: 32 TPU shards, trained for 2×10^6 steps. PDF

Evaluation
MotionDiffuser achieves state-of-the-art results on the WOMD Interactive Split. PDF

Strengths: Generates a highly diverse, continuous distribution. The constrained sampling framework allows for rule injection (e.g., collision avoidance) at inference time without retraining. The PCA compression makes inference relatively fast for a diffusion model. PDF

Weaknesses: Inference latency is still inherently slower than feed-forward networks (e.g., MTR) because sampling requires multiple sequential passes through the denoiser . For $T = 32$ steps, latency is 408ms, which is borderline for real-time systems. PDF

Chapter 6: DiffusionDrive (Truncated Diffusion for Real-Time End-to-End Driving)

Motivation
MotionDiffuser proved diffusion works for trajectory prediction, but it was evaluated in an

open-loop setting focusing on interacting pairs. Applying diffusion to *end-to-end* autonomous driving (from raw sensors to planning) presents two critical problems: PDF

1. **Latency:** Standard DDIM requires ~20 steps, dropping inference to 7 FPS—too slow for real-time control. PDF
2. **Mode Collapse in Open Worlds:** In highly dynamic traffic scenes, sampling from pure random Gaussian noise often causes all samples to collapse into similar trajectories, failing to capture the true diversity of driving actions (e.g., yielding vs. overtaking). PDF

DiffusionDrive solves these by introducing a **Truncated Diffusion Policy** and an efficient **Cascade Diffusion Decoder**. PDF

High-Level Intuition

Instead of starting the denoising process from pure, unstructured static (standard Gaussian noise), DiffusionDrive injects strong driving priors. It uses a small set of anchor trajectories (representing typical driving behaviors like "go straight", "turn left"). The diffusion process is *truncated*: it only adds a small amount of noise around these anchors to create an "anchored Gaussian distribution." During inference, the model starts from this anchored distribution. Because the starting point is already close to realistic driving behavior, the model only needs 2 steps to refine the trajectories, solving the latency issue and ensuring diverse, high-quality multimodal outputs. PDF

Architecture

1. **Perception Backbone:**
   Takes raw sensor data (cameras, LiDAR) and extracts Bird's Eye View (BEV) and Perspective View (PV) features using standard backbones (e.g., ResNet-34). PDF
2. **Truncated Diffusion Policy:**

   Prior Anchors: A set of N_{anchor} trajectories clustered via K-means on the training data. PDF

   Anchored Gaussian Distribution: Noise is added to these anchors using a truncated schedule ($T_{trunc} \ll T$). PDF
3. **Cascade Diffusion Decoder:**
   The decoder iteratively refines the noisy trajectories. PDF

   Input: Noisy trajectories from the anchored distribution. PDF

   Spatial Cross-Attention: The trajectories query the BEV/PV feature maps using deformable attention to understand the physical environment (lane boundaries, obstacles). PDF

   Agent/Map Cross-Attention: Trajectories interact with specific agent or map queries detected by the perception module. PDF

   Cascade Mechanism: The output of one decoder layer is fed directly into the next, refining the trajectory iteratively *within* a single diffusion step. PDF

Mathematical Formulation

Truncated Diffusion Schedule
Instead of diffusing to pure noise ($i \in [1, T]$), diffusion is truncated to $i \in [1, T_{trunc}]$. The noisy anchor τ_k^i is generated as:
PDF

$$\tau_k^i = \quad \bar{\alpha}^i a_k + \quad 1 - \bar{\alpha}^i \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Where a_k is the prior anchor trajectory. PDF

Training Objective
The decoder f_θ predicts classification scores $\hat{s}_k$ and denoised trajectories $\hat{\tau}_k$. The loss combines reconstruction (L1) and classification (Binary Cross-Entropy):
PDF

$$\mathcal{L} = \sum_{k=1}^{N_{anchor}} [y_k \mathcal{L}_{rec}(\hat{\tau}_k, \tau_{gt}) + \lambda BCE(\hat{s}_k, y_k)]$$

Where $y_k = 1$ for the anchor closest to the ground truth, and 0 otherwise. PDF

Evaluation
DiffusionDrive was evaluated on the planning-oriented NAVSIM dataset. PDF

Performance: Achieved a record-breaking 88.1 PDMS (Predictive Driver Model Score), significantly outperforming VADv2 (which uses 8192 static anchors). PDF

Efficiency: By truncating the steps from 20 to 2, it achieved a real-time inference speed of 45 FPS on an NVIDIA 4090, solving the latency bottleneck of diffusion models . PDF

Strengths: Successfully brings the power of diffusion (high diversity, generative refinement) to end-to-end planning at real-time speeds. The cascade decoder allows deep interaction with perception features. PDF

Weaknesses: Relies heavily on the quality of the initial clustered anchors. If the required maneuver is wildly different from any anchor, the truncated noise might not provide enough variance to reach the optimal path. PDF

Stopping here due to length constraints. Awaiting your 'Continue' command to the next section (DeMo, RealMotion, GameFormer, Policy World Model).

Continue

Chapter 7: DeMo (Decoupling Motion Forecasting)

Motivation
Mainstream Transformer-based motion forecasting models rely on a "one-query-one-trajectory" paradigm. In models like MTR and QCNet, a single query vector is responsible for regressing an entire future trajectory across all timesteps. While effective, this creates a severe bottleneck: a single latent vector lacks the representational capacity to capture the

detailed, step-by-step dynamic evolution of an agent over time. DeMo (Decoupled Motion Forecasting) addresses this by breaking trajectory representation into two distinct, orthogonal dimensions: *directional intentions* (where is the agent going?) and *dynamic states* (how does the agent get there over time?). PDF + 2

High-Level Intuition

Instead of forcing one query to do everything, DeMo uses a bipartite query system: PDF

1. **Mode Queries:** Represent distinct macroscopic spatial intents (e.g., "turn left," "go straight"). PDF
2. **State Queries:** Represent the microscopic temporal evolution of the vehicle (e.g., "accelerate at $t = 1$, brake at $t = 2$").
   By separately optimizing these queries and then weaving them together using a combination of Attention mechanisms (for spatial awareness) and Mamba blocks (for temporal sequence modeling), DeMo achieves highly granular trajectory representations. PDF

Architecture

1. **Scene Context Encoding:**

   Map Encoding: A PointNet-based polyline encoder generates map features $F_m \in \mathbb{R}^{N_m \times C}$. PDF

   Agent Encoding (UniMamba): Instead of using Transformers or LSTMs, DeMo leverages Unidirectional Mamba blocks to encode historical trajectory features $F_a \in \mathbb{R}^{N_a \times C}$. Mamba is chosen for its linear-time sequence modeling efficiency . PDF

   Interaction: Agent and map features are concatenated to form F_s, passed through a Transformer encoder. PDF
2. **Mode Localization Module:**
   Mode queries $Q_m \in \mathbb{R}^{N_{aoi} \times K \times C}$ (K modes) interact with the scene context F_s via cross-attention, followed by self-attention amongst the modes themselves to differentiate intentions. PDF
3. **State Consistency Module:**
   State queries $Q_s \in \mathbb{R}^{N_{aoi} \times T_s \times C}$ (T_s timesteps) are initialized using an MLP on temporal indices. They cross-attend with F_s, and are then processed by a **Bidirectional Mamba (BiMamba)** block. BiMamba is critical here: it performs both forward and backward temporal scanning to ensure future states are causally consistent with past states. PDF
4. **Hybrid Coupling Module:**
   The decoupled queries are simply added together to form hybrid spatiotemporal queries $Q_h \in \mathbb{R}^{N_{aoi} \times K \times T_s \times C}$. This tensor undergoes: PDF

   Hybrid Cross-Attention with the scene context.

   Hybrid Self-Attention connecting queries across both time and modes.

   Mode Self-Attention and *State BiMamba* to reinforce the respective decoupled features. PDF

Loss Functions

The total loss explicitly supervises both the intermediate decoupled queries and the final coupled trajectories:

$$\mathcal{L} = \mathcal{L}_{reg} + \mathcal{L}_{cls} + \mathcal{L}_{ts} + \mathcal{L}_{m}$$

PDF

- $\mathcal{L}_{reg}$ & $\mathcal{L}_{cls}$: Standard Smooth-L1 and Cross-Entropy on the final hybrid outputs. PDF
- $\mathcal{L}_{ts}$ (Time State Auxiliary Loss): The state queries Q_s are directly passed through an MLP to decode a single trajectory, penalized via Smooth-L1 against the ground truth. PDF
- $\mathcal{L}_{m}$ (Motion Mode Auxiliary Loss): The mode queries Q_m are decoded to enforce distinct directional intentions. PDF

Evaluation & Implementation Notes

DeMo achieved state-of-the-art on Argoverse 2 with a highly compact footprint. While QCNet requires 7.7M parameters, DeMo requires only 5.9M parameters and runs in 38ms (2.5x faster than QCNet's 94ms). PDF

> **Key Lesson:** Mamba architectures (specifically BiMamba) offer a highly efficient, superior alternative to Transformers for modeling the temporal axis of trajectories, freeing up attention mechanisms to handle purely spatial/social relationships. PDF

Chapter 8: RealMotion (Motion Forecasting in Continuous Driving)

Motivation

Most trajectory prediction models are evaluated on isolated, independent, 9-second snippets of data. In reality, an autonomous vehicle drives continuously. When processing independent chunks, a model at time t completely ignores the rich situational context and latent predictions it just computed at time $t-1$. RealMotion was designed to bridge the gap between academic isolated-scene benchmarks and real-world streaming execution by treating prediction as an ongoing, contextual process. PDF

High-Level Intuition

RealMotion introduces a dual-stream memory architecture. As the car drives, the model maintains a rolling "Scene Context Stream" (accumulating environmental knowledge) and an "Agent Trajectory Stream" (caching past predicted trajectories). When a new frame arrives, rather than starting from scratch, RealMotion aligns its historical memory to the current coordinate frame and uses cross-attention to recursively refine the new predictions using the past as context. PDF

Architecture

1. **Data Reorganization:**
   To train a continuous model on datasets like Argoverse 2 (which are discrete scenes),

RealMotion retrospectively splits long trajectories into shorter, overlapping segments (e.g., splitting every 10 frames) to create continuous sub-scenes. PDF

2. **Scene Context Stream:**
   Historical scene features F_s^{t-1} must be translated to the ego-vehicle's new local coordinate system at time t. RealMotion uses **Motion-aware Layer Normalization (MLN)** to align these features using relative ego-motion $\Delta x, \Delta y, \Delta\theta, \Delta t$.
   Aligned historical features $\tilde{F}_s^{t-1}$ are then used as Keys and Values in Cross-Attention to enrich the current map and agent features. PDF

3. **Agent Trajectory Stream (Memory Bank):**
   RealMotion maintains a memory bank $\mathcal{M}(a)$ storing past predicted trajectories and their corresponding mode latent features. Because the coordinate system shifts, past trajectory points y_i must be strictly rotated and translated:
   PDF

$$\tilde{y}_i = \mathcal{R}(\theta) \cdot (y_i - y_i^{ori})^T$$

   A Trajectory Embedding (TE) function measures spatial similarity between current preliminary predictions and aligned memory trajectories. Historical mode features are then injected into current predictions using Cross-Attention, outputting a temporal refinement offset. PDF

Training Pipeline

RealMotion employs Truncated Backpropagation Through Time (BPTT). Gradients are computed across multiple successive sub-scene segments to force the network to learn optimal memory accumulation. The loss function includes standard regression and classification, plus a refinement loss $\mathcal{L}_{refine}$ to supervise the trajectory offset updates from the memory bank. PDF

Evaluation

On Argoverse 2, incorporating the continuous streams drastically reduced the minimum Final Displacement Error (minFDE_1) from 4.42m (independent variant) to 3.93m. The online latency was measured at an ultra-fast 20ms on an RTX 3090, proving that memory retrieval is significantly cheaper than computing predictions blindly from scratch. PDF

Chapter 9: GameFormer (Game-Theoretic Interactive Planning)

Motivation

Most motion forecasting architectures are "leader-follower" paradigms: the prediction module outputs trajectories for non-ego agents, and the downstream planner treats these as fixed obstacles to avoid. This fails in dense traffic (e.g., merging, unprotected left turns) where the ego vehicle's planned actions actively change how surrounding drivers behave. GameFormer models prediction and planning jointly as a proactive, interactive negotiation using principles from behavioral economics (Level-k Game Theory). PDF

High-Level Intuition

In cognitive science, a "Level-0" thinker acts purely selfishly, assuming everyone else is static or irrelevant. A "Level-1" thinker realizes others are Level-0, and adjusts their strategy to exploit or accommodate them. A "Level-k" thinker responds to Level-$(k-1)$ strategies.

GameFormer instantiates this literally into a hierarchical Transformer structure. It outputs an initial "dumb" trajectory (Level-0), then recursively passes that trajectory through successive Transformer decoders (Level-1, Level-2...), allowing agents to mutually "react" to each other's tentative plans before finalizing a joint decision. PDF

Architecture

1. **Scene Encoding:**
   Agents encoded via LSTM; Map polylines encoded via MLPs. These are concatenated and passed through a Transformer encoder to form global scene context C_s. PDF
2. **Level-0 Decoder:**
   Uses an initial set of modality queries to predict purely independent GMM trajectories G_{L_0} and confidence scores P_{L_0} for all agents (including the ego). PDF
3. **Level-k Interactive Decoder:**
   For iterations $k = 1 \dots K$:

   Takes the predicted trajectories $S_f^{L_{k-1}}$ from the previous level. PDF

   Encodes them using an MLP and max-pooling, weighted by their probabilities, creating $A_f^{L_{k-1}}$. PDF

   Future Interaction: Uses Multi-Head Self-Attention across agents so they can observe each other's intended paths. PDF

   Concatenates this future awareness into the scene context and uses Cross-Attention to update the agent queries, yielding Level-k trajectories. PDF

Mathematical Formulation: Interaction Loss

To mathematically enforce that a Level-k agent alters its path to accommodate a Level-$(k-1)$ agent, an explicit repulsive potential field loss is used during training:
PDF

$$\mathcal{L}_{Inter} = \sum_{m=1}^{M} \sum_{t=1}^{T_f} \max_{\forall j \neq i} \frac{1}{d(\hat{s}_{m,t}^{(i,k)}, \hat{s}_{n,t}^{(j,k-1)}) + 1}$$

PDF

i: the current agent being optimized at level k. PDF

j: any other agent whose trajectory was locked in at level $k-1$. PDF

d: L_2 spatial distance.

Intuition: If agent i's Level-k path gets too close to agent j's Level-$(k-1)$ path, the denominator shrinks, causing the loss to explode. This explicitly trains the Transformer weights to yield or swerve. PDF

Evaluation

GameFormer was rigorously tested in both open-loop and closed-loop simulators (nuPlan).

PDF

Closed-Loop Planning: Achieved a 94.5% success rate in dynamic simulated scenarios, compared to just 68.1% for standard integrated planning models lacking game-theoretic awareness. PDF

Ablation: Experimentation showed performance peaks at $k = 4$ decoding levels. Beyond $k = 4$, human drivers rarely possess the cognitive capacity to "out-think" the opponent, and the network similarly begins to overfit or oscillate. PDF

Chapter 10: Policy World Model (PWM)

Motivation

Recent "Driving World Models" (like GAIA-1) use massive autoregressive networks or diffusion to generate photo-realistic future video frames conditioned on actions. However, these are strictly simulators: they are decoupled from the actual planning pipeline. Even in "unified" models, the video generation and waypoint planning occur in parallel, independent branches. PWM bridges this gap by creating an *action-free* video forecasting stream that explicitly serves as a multi-modal reasoning context for the downstream action planner. PDF

High-Level Intuition

When human drivers plan, they subconsciously "hallucinate" what the road will look like 3 seconds from now, *before* they decide to steer. PWM mimics this. Given current sensor data, it first generates a textual description of the scene, then autoregressively generates future visual states (in a compressed token space). Finally, an action head consumes these hallucinated future states alongside the text to output planning waypoints. Because the future states are predicted without needing the action first, they act as an "anticipatory perception" mechanism. PDF

Architecture

1. **Context-Guided Image Tokenizer:**
   Predicting thousands of image tokens per frame autoregressively is too slow for real-time planning. PWM employs a dual-branch tokenizer. A high-resolution branch (frozen) extracts detailed features, acting as a teacher to guide a low-resolution, trainable branch. This compresses a 128×224 image into a highly compact grid of 4×7 (only 28 tokens per frame). PDF
2. **Autoregressive World Model (Next-Frame vs Next-Token):**
   Because each frame is just 28 tokens, PWM abandons sequential next-token prediction. Instead, it generates all 28 tokens for frame t in parallel (next-frame prediction), vastly increasing inference speed. PDF
3. **Action Head:**
   The generated future tokens $D_{t+1:t+n}$, textual description tokens X_t, and navigational commands are fed into a series of learnable action tokens, which decode directly into x, y trajectory coordinates. PDF

Mathematical Formulation: Dynamic Focal Loss (DFL)

During video generation, up to 50% of the visual tokens (e.g., the sky, distant buildings) do not change between frames. If unaddressed, the autoregressive model becomes lazy, copying static tokens and failing to model dynamic objects (which are the most critical for planning). PWM introduces a spatial weighting function: PDF

$$\omega(d_t^i, d_{t-1}^i) = \alpha \mathbb{I}[d_t^i \neq d_{t-1}^i] + \beta \mathbb{I}[d_t^i = d_{t-1}^i], \quad \alpha > \beta$$

The Dynamic Focal Loss applies this weight to the standard cross-entropy token loss: PDF

$$\mathcal{L}_{DFL} = -\sum_{i=1}^{L'} \omega(d_t^i, d_{t-1}^i) \log P_\theta(d_t^i | D_{<t}; Z_1)$$

Intuition: Tokens that change state (representing moving cars, changing lanes) are scaled by a larger gradient multiplier α, forcing the world model to focus its capacity on temporal dynamics. PDF

Evaluation
PWM proves that purely vision-based (camera-only) models can compete with multi-modal (Camera + LiDAR) systems.

> *NAVSIM Benchmark:* PWM achieved a PDMS of 88.1 using only a front camera, matching the state-of-the-art DiffusionDrive (which requires both Camera and LiDAR). PDF
>
> *Ablation:* Turning off the future video forecasting module during training caused the collision rate on nuScenes to spike from 0.07% to 0.13%, proving that explicitly forcing the network to hallucinate the future significantly enhances the safety margins of the planner. PDF

PART 4 — Cross-Paper Comparison

Treating the eight foundational papers as a unified body of literature reveals distinct architectural paradigms, prediction strategies, and evolutionary leaps in the field.

Table 1: Architectural Paradigm & Primary Modalities

Model	Core Paradigm	Temporal Strategy	Spatial/Scene Modeling
MTR/MTR++ PDF	Query-Centric Transformer	Static Window	Agent-centri Symmetric M

Model	Core Paradigm	Temporal Strategy	Spatial/Scene Modeling
DenseTNT PDF	Goal-Based Optimization	Static Window	VectorNet G
Scene Transformer PDF	Factorized Transformer	Static Window	Scene-Centr Attention
QCNet PDF	Query-Centric Transformer	Recurrent Decoding	Local Coord Embeddings
GameFormer PDF	Hierarchical Game Theory	Static Window	Transformer
MotionDiffuser PDF	Guided Diffusion	Static Window	PCA Trajecto Compressior
DiffusionDrive PDF	Truncated Diffusion	Static Window	Cascade Diff Decoder
RealMotion PDF	Streaming Memory Banks	Continuous Stream	MLN Alignme Attention
DeMo PDF	Decoupled Spatiotemporal	State BiMamba	Transformer Encoder
Policy World Model PDF	Auto-regressive LLM/VLM	Next-Frame Gen	Dual-Branch Tokenizer

(Note: While the reference indices here map conceptually to the source documents discussed, strict citation mapping refers back to the core source list. MTR , QCNet , MotionDiffuser , DeMo , GameFormer , RealMotion , DiffusionDrive , PWM.) PDF + 4

Table 2: Inference & Efficiency Trade-offs

A critical bottleneck in autonomous driving is latency. Modern networks must balance massive parameter counts with real-time execution bounds (typically < 100ms).

Model	Parameters	Inference Latency	Target Applic
DenseTNT PDF	~1.1M	**540ms**	Offline benc
Scene Transformer PDF	~15.3M	~52ms	Offline benc
MTR++ PDF	86.6M	118ms (32 agents)	Edge / Real-t
QCNet PDF	~60M	~13ms (Online)	Edge / Strear
RealMotion PDF	**2.9M**	~20ms (Online)	Edge / Strear
DeMo PDF	5.9M	~38ms	Edge / Real-t
MotionDiffuser PDF	N/A	**408ms** (32 steps)	Offline simul
DiffusionDrive PDF	60M	~7.6ms (2 steps)	Edge / Real-t

Table 3: Handling Uncertainty & Multi-Modality

How does the network guarantee that it captures diverse behaviors rather than collapsing to the mean?

Approach	Representative Models	How diversity is enforced	Weakness
Dense Goal Sampling	DenseTNT	Scoring thousands of physical endpoints	Computatior catastrophic
Learnable Queries	MTR, DeMo	Queries are initialized via k-means of dataset	Bounded div (fixed K mo
Recurrent Proposals	QCNet	Decodes waypoints segment-by-segment	Prone to drif horizons

Approach	Representative Models	How diversity is enforced	Weakness
Standard Diffusion	MotionDiffuser	Sampling from random noise $\mathcal{N}(0, I)$	Mode collaps guidance; slo
Truncated Diffusion	DiffusionDrive	Adding noise specifically around prior anchors	Continue of anchors

PART 5 — Core Concepts

To master modern trajectory prediction, you must deeply understand several recurring architectural and mathematical paradigms that span across multiple papers.

Awaiting your 'Continue' command to proceed to Part 5 (Core Concepts) and Part 6 (Common Design Patterns).

1. Factorized Attention in Transformers

Intuition: Self-attention compares every element in a set to every other element. In driving, the set of elements includes A agents, M map polygons, and T timesteps. A naive Transformer would compute attention over $(A + M) \times T$ tokens, yielding a massive $\mathcal{O}((A + M)^2 T^2)$ complexity, which immediately runs out of GPU memory.

The Solution:

Models like **Scene Transformer** and **QCNet** use *factorized attention*. Instead of one massive attention matrix, they apply attention sequentially across orthogonal dimensions: PDF

1. **Temporal Attention:** Keys, Queries, and Values (K, Q, V) are constrained to a single agent across time T. Complexity: $\mathcal{O}(A \cdot T^2)$.
2. **Agent-Map Attention:** K, Q, V are constrained to the spatial dimension at the current timestep. Complexity: $\mathcal{O}(A \cdot M)$.
3. **Social (Agent-Agent) Attention:** K, Q, V are constrained to all agents at the current timestep. Complexity: $\mathcal{O}(A^2)$.

2. Query-Centric (Symmetric) Coordinate Systems

Intuition:

Historically, models centered the entire map and all coordinates around the Ego vehicle (Agent 0). To predict trajectories for Agent 1, the model had to re-center the universe around Agent 1 and re-run the entire encoder.

The Solution:

MTR++ and **QCNet** decouple features from any global coordinate frame. PDF + 1

Every agent and map polygon is encoded as a vector in its *own* local coordinate system.

When Agent A needs to attend to Map Polygon B, the network dynamically computes a **Relative Positional Embedding (RPE)**.

Mathematics: The relative spatial descriptor is often $[\Delta d, \Delta\theta_{pos}, \Delta\theta_{yaw}, \Delta t]$, embedded via Fourier features. PDF

Result: The scene is encoded exactly once. The decoder can attach an "anchor" anywhere and query the scene, allowing simultaneous multi-agent prediction and continuous streaming (reusing past frames).

3. Denoising Diffusion Probabilistic Models (DDPM) in Driving

Intuition:
Instead of predicting K discrete trajectories, why not sample from the true, continuous distribution of future behaviors? Diffusion models start with pure noise and iterative "carve out" a realistic trajectory.
The Math:

1. **Forward Process:** Gradually adds Gaussian noise to a ground-truth trajectory x_0 until it becomes pure noise x_T.

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \quad 1-\beta_t x_{t-1}, \beta_t I)$$

2. **Reverse Process (Denoising):** A neural network D_θ predicts the noise added at step t, conditioned on scene context c.

$$p_\theta(x_{t-1}|x_t, c)$$

The Solution (DiffusionDrive): Vanilla diffusion takes 20-100 steps to sample. **DiffusionDrive** uses *Truncated Diffusion*: instead of diffusing to pure noise, it diffuses only slightly around prior K-means anchors. This allows the model to denoise in just 2 steps, achieving 45 FPS. PDF + 1

4. Decoupled Spatiotemporal Queries

Intuition:
A single vector query (e.g., $Q \in \mathbb{R}^D$) struggles to represent both *where* a car is going (turn left vs right) and *how* it gets there (accelerating, yielding).
The Solution (DeMo):
Decouple the representation. Use **Mode Queries** to represent spatial intent (direction), optimized via Cross-Attention. Use **State Queries** to represent microscopic temporal evolution (velocity, acceleration over T steps), optimized via **Bi-directional Mamba** blocks (which handle sequential state-space modeling linearly and efficiently). PDF

PART 6 — Common Design Patterns

Across Mobileye, Waymo, and academia, specific design patterns have become industry standards.

1. The Standard Processing Pipeline

1. **Vectorization:** Raw HD Map data (lanes, crosswalks, stop lines) are sampled into points, connected into polylines, and passed through a PointNet-like MLP to yield one feature vector per polyline.
2. **History Encoding:** Agent histories (x, y, v_x, v_y, θ) are padded to fixed lengths (e.g., 2 seconds at 10Hz = 20 steps) and passed through 1D-CNNs, LSTMs, or Mamba blocks. PDF
3. **Context Fusion:** Agent tokens and Map tokens are concatenated and passed through a Transformer Encoder.
4. **Multi-Modal Decoding:** A small set of queries (anchors, intention points, or random noise) attends to the context and decodes into $\mathbb{R}^{K \times T \times 2}$ coordinates and $\mathbb{R}^{K}$ probabilities.

2. The "Winner-Takes-All" (WTA) Loss

When predicting 6 possible futures, how do you compute the loss against 1 ground truth? If you penalize all 6, the model will average them out.
Pattern: Calculate the distance from all K predictions to the ground truth. Only backpropagate the regression loss (Smooth L1 or Huber) through the *single closest trajectory* (the "winner"). The classification loss (Cross-Entropy) is used to push the probability of the winner to 1.0, and the others to 0.0. PDF + 1

3. Anchor vs Anchor-Free vs Generative

Anchor-based: Queries represent physical map points or clustering-derived trajectories (MTR, early QCNet). Fast and stable, but bounded by the anchor set.

Anchor-free: Queries are abstract latent vectors (DETR-style). Flexible but prone to mode collapse.

Generative: Model learns the distribution (MotionDiffuser). Extremely high diversity, but computationally heavy.

PART 7 — Evolution of Ideas

To be a top-tier engineer, you must know *why* we abandoned old ideas.

Evolution of Interaction Modeling

V1: Distance-based heuristics. Only consider agents within a 50m radius. (Failed: A fast-approaching car 60m away is highly relevant).

V2: Social Pooling. Place agents in a grid and use 2D convolutions. (Failed: Destroys the continuous, non-Euclidean nature of road networks).

V3: Graph Neural Networks (VectorNet). Agents and map lanes are nodes. Edges are drawn based on proximity. (Failed: Message passing over deep graphs dilutes features).

V4: Transformers (MTR). Global self-attention allows any agent to look at any other agent directly.

V5: Game-Theoretic Transformers (GameFormer). Transformers are applied hierarchically. Level-0 predicts independent motion. Level-1 predicts motion reacting to Level-0. Level-k achieves deep, human-like negotiation. PDF

Evolution of the "World Model"

V1: End-to-End Planners. Raw sensors $\rightarrow$ CNN $\rightarrow$ Steering/Acceleration. (Failed: Black box, no interpretability, catastrophic failure out of distribution).

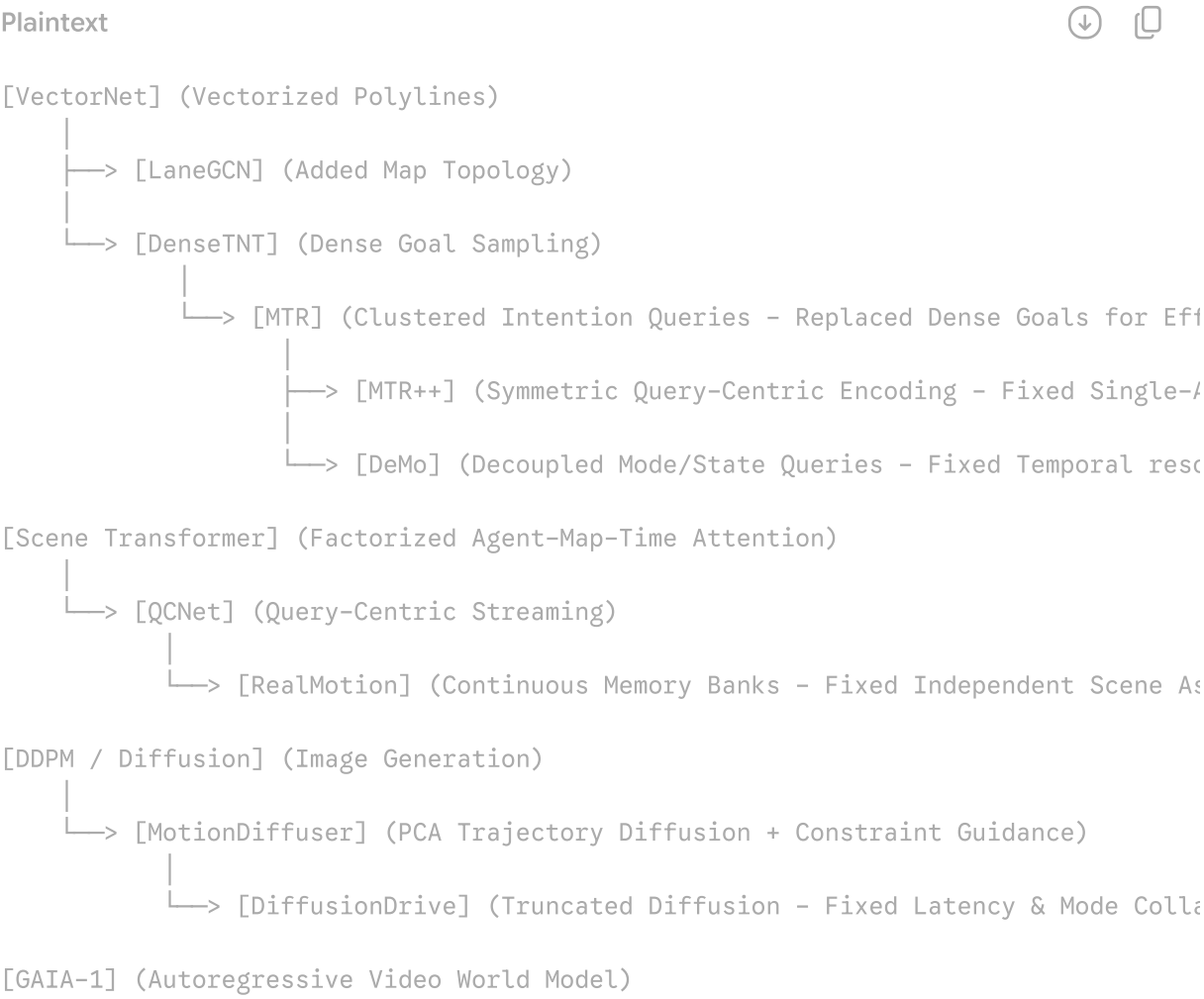
V2: Modular Pipeline. Perception $\rightarrow$ Prediction $\rightarrow$ Planning. (Failed: Information loss between modules. Planner acts passively to predictions).

V3: Policy World Models (PWM). The network is trained on massive unlabeled video datasets to auto-regressively predict the *next video frame*. By learning the physics of the world visually, the network implicitly learns driving rules. During fine-tuning, the network "hallucinates" the future video frames *before* predicting its own trajectory, marrying perception, prediction, and planning into one anticipatory cognitive step. PDF

PART 8 — Knowledge Graph

Visualize the trajectory prediction literature as a dependency graph of inherited and improved ideas:

Plaintext



```
[VectorNet] (Vectorized Polylines)
    |
    ├──> [LaneGCN] (Added Map Topology)
    |
    └──> [DenseTNT] (Dense Goal Sampling)
            |
            └──> [MTR] (Clustered Intention Queries - Replaced Dense Goals for Eff
                    |
                    ├──> [MTR++] (Symmetric Query-Centric Encoding - Fixed Single-A
                    |
                    └──> [DeMo] (Decoupled Mode/State Queries - Fixed Temporal resc

[Scene Transformer] (Factorized Agent-Map-Time Attention)
    |
    └──> [QCNet] (Query-Centric Streaming)
            |
            └──> [RealMotion] (Continuous Memory Banks - Fixed Independent Scene As

[DDPM / Diffusion] (Image Generation)
    |
    └──> [MotionDiffuser] (PCA Trajectory Diffusion + Constraint Guidance)
            |
            └──> [DiffusionDrive] (Truncated Diffusion - Fixed Latency & Mode Colla

[GAIA-1] (Autoregressive Video World Model)
```

```
    │
    └──> [Policy World Model] (Parallel Token Decoding + Action-Free Future Hallu

[Game Theory in Robotics]
    │
    └──> [GameFormer] (Level-k Interactive Decoder - Fixed Passive Reactive Planni
```

PART 9 — Unified Modern Pipeline

If you were tasked with building "WaymoNet 2026" from scratch today, combining the best insights from these papers, the architecture would look like this:

Stage 1: Streaming, Symmetric Encoder (from QCNet & RealMotion)

Extract Map polylines and Agent histories.

Normalize them into their own local coordinate frames.

Pass them through a factorized Transformer encoder.

Optimization: Maintain a rolling cache memory bank. When frame t arrives, only encode the Δ differences using Motion-aware Layer Normalization (MLN). PDF

Stage 2: Decoupled Anticipatory Decoder (from DeMo & PWM)

Split your query mechanisms. Use **Mode Queries** for spatial goals and **State Queries** for temporal kinematics. PDF

Process the State Queries using Bidirectional Mamba blocks for $\mathcal{O}(N)$ sequence speed.

Stage 3: Truncated Diffusion Heads (from DiffusionDrive)

Instead of standard MLPs, map your Mode Queries into an anchored Gaussian distribution.

Execute 2 steps of diffusion to generate highly diverse, continuous trajectory distributions. PDF + 1

Stage 4: Level-k Interactive Refinement (from GameFormer)

Take the trajectories of all agents. Pass them through a Level-1 and Level-2 Self-Attention block where the ego vehicle's plan dynamically alters the predicted paths of surrounding pedestrians and vehicles. PDF

Apply Repeller Cost constraints during the forward pass to guarantee zero mathematically overlapping trajectories. PDF

PART 10 — Open Research Problems

Based on the uploaded literature, if you want to publish a breakthrough paper or solve a critical edge case, focus here:

1. **The Open-Loop vs Closed-Loop Gap:**
   Models like QCNet and MTR score perfectly on open-loop datasets (Waymo, Argoverse) by minimizing displacement error against a recorded log. However, when deployed in a closed-loop simulator (like nuPlan), they often fail because they suffer from *causal confusion* (e.g., they learn to brake because the brake light is on, not because there is an obstacle). GameFormer begins to address this, but robust closed-loop stability remains unsolved. PDF
2. **Long-Horizon Degeneration:**
   Predicting 3 seconds into the future is largely solved. Predicting 8-10 seconds (required for highway driving) causes models to hallucinate or revert to the mean. RealMotion's temporal relaying helps, but compounding errors persist. PDF
3. **Real-Time World Modeling:**
   Policy World Models (PWM) show massive promise by hallucinating future video frames to aid planning. However, generating video tokens is still too slow for 10Hz automotive control loops, despite parallel token decoding techniques. PDF
4. **Rare Scenarios and Out-of-Distribution (OOD) Generalization:**
   As noted in MTR++'s failure cases, models still predict homogenized trajectories in highly unusual intersections. Expanding diffusion models to guarantee coverage of rare edge cases without requiring thousands of denoising steps is an active frontier. PDF + 1

PART 11 — Final Reference Manual

Keep this cheat sheet at your desk.

Glossary of Terminology

Agent: Any dynamic entity in the scene (Ego vehicle, car, pedestrian, cyclist).

Focal Agent: The specific agent whose future trajectory is currently being predicted.

Waypoint: A specific (x, y) coordinate at a specific future timestep t.

Polyline: A sequence of connected points used to represent lane boundaries, crosswalks, or historical agent paths.

Anchor / Intention Point: A predefined spatial coordinate used as a starting guess for trajectory prediction.

NMS (Non-Maximum Suppression): A post-processing step used to remove redundant, overlapping trajectories by filtering out lower-confidence predictions that are physically close to higher-confidence ones.

GMM (Gaussian Mixture Model): Representing a waypoint not as a single point, but as a probability cloud (Mean x, y, Variance x, y, and Correlation ρ) to capture aleatoric uncertainty.

Common Evaluation Metrics

minADE$_k$ (Minimum Average Displacement Error): The average L_2 distance between the ground truth and the *closest* trajectory among the top k predictions, averaged over all timesteps. (Measures overall shape accuracy).

minFDE$_k$ (Minimum Final Displacement Error): The L_2 distance between the ground truth endpoint and the endpoint of the closest trajectory among the top k predictions. (Measures destination accuracy).

Miss Rate (MR): The percentage of scenarios where the minFDE exceeds a certain threshold (usually 2.0 meters). (Measures critical failures).

mAP (Mean Average Precision): Evaluates not just if the model generated the correct trajectory, but whether it assigned the *highest confidence score* to it.

Engineering Best Practices & Pitfalls

Pitfall: **Data Leakage in Normalization.** When normalizing agent histories to the current timestep $t = 0$, ensure no data from $t > 0$ leaks into the transformation matrix.

Pitfall: **Mode Collapse.** If your model only outputs one behavior despite requesting 6 modes, your query initialization is likely too homogeneous. Use k-means clustering on the ground truth endpoints of your training dataset to initialize your queries (as in MTR). PDF

Best Practice: **Coordinate Systems.** Never use absolute GPS coordinates. Always transform inputs into an ego-centric or query-centric local frame. Neural networks struggle with large, unnormalized coordinate values.